

# VTA ComputeModule — Complete Dataflow Guide

Covers: HLS original → SystemC translation, all variables explained, flowcharts for every stage, complete code listing, and a worked example.

## 1. Big Picture: What Does the Compute Stage Do?

In VTA, three hardware units run in parallel — **Load**, **Compute**, **Store**. Each gets its own stream of instructions. The Compute stage is the mathematical core:

| Operation | What it does                                  | Example use in ResNet  |
|-----------|-----------------------------------------------|------------------------|
| GEMM      | Matrix-multiply: weight × input → accumulator | Convolution layer      |
| ALU       | Element-wise ops: ReLU (MAX 0), shift, add    | Activation, batch-norm |
| LOAD ACC  | Load bias values into accumulator SRAM        | Bias initialisation    |
| NOP       | No operation — pipeline padding               | —                      |
| FINISH    | End of layer — signal ARM that layer is done  | Every layer boundary   |

The Compute stage does *not* physically move data from DRAM. Load already put data into SRAM buffers (inp\_mem, wgt\_mem). Compute reads those buffers, calculates, writes to out\_mem. Store then copies out\_mem → DRAM.

## 2. HLS Original vs SystemC — Side-by-Side Comparison

### HLS (vta.cc) — Sequential function, one-shot execution

The `compute()` function in HLS is called once per instruction. It blocks until dependencies are satisfied, runs the math, pushes tokens, and returns. The FPGA synthesiser turns loops into pipelined hardware.

### SystemC — Event-driven `sc_module`, always running

`ComputeModule` never "returns". Instead, it reacts to events (signals firing, events notified). Each stage of the pipeline is a separate `SC_METHOD`. The SystemC scheduler calls the right method when its event fires.

| HLS concept                                                        | SystemC equivalent                                                                   | Why the change?                                                    |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>hls::stream&lt;insn_T&gt;</code><br><code>gemm_queue</code>  | <code>Queue</code> <code>sc_module</code> with <code>vld/rdy/data/end</code> signals | SystemC has no stream — we model the handshake protocol explicitly |
| <code>hls::stream&lt;bool&gt;</code><br><code>l2g_dep_queue</code> | <code>l2c</code> <code>Queue</code> module ( <code>sc_signal</code> wires)           | Same idea — token passing, but hardware-accurate timing            |
| Function call blocks until dep satisfied                           | <code>is_waiting_prev</code> flag + event from VLD rising edge                       | SystemC can't block — must remember waiting state and resume       |

|                                                    |                                                                                                                              | on event                                                                                    |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>done = 1</code> on FINISH                    | <code>out_trig</code> signal → ARM module                                                                                    | ARM is an independent <code>sc_module</code> ;<br>signal is how modules communicate         |
| <code>gemm()</code> / <code>alu()</code> functions | <code>latency()</code> returns <code>sc_time</code> , then<br><code>finish.notify(latency())</code>                          | Simulator doesn't do the math — it<br>just advances simulated time by the<br>correct amount |
| <code>g2l_dep_queue.write(1)</code>                | Assert <code>push_prev_vld</code> , wait for<br><code>push_prev_rdy</code> , send data, assert<br><code>push_prev_end</code> | Full handshake needed for<br>hardware-accurate flow control                                 |

## 3. Every Variable in ComputeModule — Explained

### 3.1 Ports (signals going in/out of the module)

| Variable       | Type               | Direction           | What it means                                                                  |
|----------------|--------------------|---------------------|--------------------------------------------------------------------------------|
| out_trig       | sc_out<bool>       | OUT → ARM           | Goes HIGH for one tick to tell ARM "this layer is done, start the next one"    |
| i_queue_vld    | sc_in<bool>        | IN ← Compute Queue  | Queue says: "I have an instruction ready for you"                              |
| i_queue_rdy    | sc_out<bool>       | OUT → Compute Queue | ComputeModule says: "I am ready to receive it"                                 |
| i_queue_data   | sc_in<sc_int<64>>  | IN ← Compute Queue  | The actual instruction word (64 bits at a time)                                |
| i_queue_end    | sc_in<bool>        | IN ← Compute Queue  | Queue says: "that was the last 64-bit chunk of this instruction"               |
| pull_prev_vld  | sc_in<bool>        | IN ← l2c queue      | Load→Compute dep token available ("load finished, you can compute")            |
| pull_prev_rdy  | sc_out<bool>       | OUT → l2c queue     | Compute says "I accept this token"                                             |
| pull_prev_data | sc_in<sc_int<64>>  | IN ← l2c queue      | Token payload (the PC of the load instruction that finished)                   |
| pull_prev_end  | sc_in<bool>        | IN ← l2c queue      | Handshake complete — token fully received                                      |
| push_prev_vld  | sc_out<bool>       | OUT → c2l queue     | Compute→Load dep token being sent ("I'm done with acc buffer, load can reuse") |
| push_prev_rdy  | sc_in<bool>        | IN ← c2l queue      | c2l queue says "I'm ready to receive"                                          |
| push_prev_data | sc_out<sc_int<64>> | OUT → c2l queue     | Token payload (Compute's PC)                                                   |
| push_prev_end  | sc_out<bool>       | OUT → c2l queue     | Handshake end — token fully sent                                               |
| pull_next_vld  | sc_in<bool>        | IN ← s2c queue      | Store→Compute dep token ("store finished, output buffer free")                 |
| pull_next_rdy  | sc_out<bool>       | OUT → s2c queue     | Compute accepts s2c token                                                      |
| pull_next_data | sc_in<sc_int<64>>  | IN ← s2c queue      | Token payload from store                                                       |
| pull_next_end  | sc_in<bool>        | IN ← s2c queue      | s2c handshake complete                                                         |
| push_next_vld  | sc_out<bool>       | OUT → c2s queue     | Compute→Store dep token ("results in out_mem, you can store")                  |
| push_next_rdy  | sc_in<bool>        | IN ← c2s queue      | c2s queue ready to receive                                                     |

|                             |                                             |                 |                        |
|-----------------------------|---------------------------------------------|-----------------|------------------------|
| <code>push_next_data</code> | <code>sc_out&lt;sc_int&lt;64&gt;&gt;</code> | OUT → c2s queue | Token payload to store |
| <code>push_next_end</code>  | <code>sc_out&lt;bool&gt;</code>             | OUT → c2s queue | c2s handshake end      |

### 3.2 Internal State Variables

| Variable                         | Type                           | What it tracks                                                                                                                                                  |
|----------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>current</code>             | Instruction*                   | Pointer to the instruction currently being processed. NULL between instructions.                                                                                |
| <code>prev_data</code>           | <code>sc_int&lt;64&gt;*</code> | Holds the data received from the l2c (Load→Compute) dependency token. Not the actual input data — it's the token payload (Load's PC). NULL until token arrives. |
| <code>next_data</code>           | <code>sc_int&lt;64&gt;*</code> | Holds the data received from the s2c (Store→Compute) dependency token. NULL until token arrives.                                                                |
| <code>is_waiting_prev</code>     | bool                           | TRUE if instruction needs pop_prev but l2c token hasn't arrived yet — module is stalled waiting                                                                 |
| <code>is_waiting_next</code>     | bool                           | TRUE if instruction needs pop_next but s2c token hasn't arrived yet                                                                                             |
| <code>out_signal</code>          | bool                           | Staging flag: set TRUE when we want to pulse out_trig. The handler reads this and drives the actual port.                                                       |
| <code>did_push_prev</code>       | bool                           | Confirms that push_prev (c2l token) handshake completed successfully                                                                                            |
| <code>did_push_next</code>       | bool                           | Confirms that push_next (c2s token) handshake completed successfully                                                                                            |
| <code>push_prev_vld_state</code> | bool                           | Desired value for push_prev_vld port — written to the port by the handler                                                                                       |
| <code>push_prev_end_state</code> | bool                           | Desired value for push_prev_end port                                                                                                                            |
| <code>push_next_vld_state</code> | bool                           | Desired value for push_next_vld port                                                                                                                            |
| <code>push_next_end_state</code> | bool                           | Desired value for push_next_end port                                                                                                                            |
| <code>pull_prev_rdy_state</code> | bool                           | Desired value for pull_prev_rdy port                                                                                                                            |
| <code>pull_next_rdy_state</code> | bool                           | Desired value for pull_next_rdy port                                                                                                                            |
| <code>current_layer</code>       | static int                     | Global counter, shared across all instructions. Increments every time a FINISH instruction is processed. Printed in "FINISH LAYER N" output.                    |
| <code>result_data</code>         | <code>sc_int&lt;32&gt;*</code> | Placeholder for computation result (always set to 5 — simulator does not actually compute GEMM/ALU math)                                                        |

### 3.3 Internal Events (sc\_event)

| Event              | Notified by                           | Triggers                         | Meaning                                      |
|--------------------|---------------------------------------|----------------------------------|----------------------------------------------|
| <code>fetch</code> | Module base class / push_dependencies | <code>fetch_instruction()</code> | "Go get the next instruction from the queue" |

|                                     |                                                                          |                                               |                                                                 |
|-------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------------|
| <code>check_dep</code>              | Module base class after instruction received                             | <code>check_dependencies()</code>             | "You have a new instruction — check if deps are satisfied"      |
| <code>receive_dep</code>            | <code>pull_prev/next_end</code> handlers                                 | <code>receive_dependencies()</code>           | "A dep token arrived — see if all required tokens are now here" |
| <code>dep_received</code>           | <code>receive_dependencies()</code> or <code>check_dependencies()</code> | <code>dependencies_received()</code>          | "All required tokens received — start the timer"                |
| <code>finish</code>                 | <code>dependencies_received()</code> after latency()                     | <code>finalize_instruction()</code>           | "Simulated compute time elapsed — instruction done"             |
| <code>push_dep</code>               | <code>finalize_instruction()</code>                                      | <code>push_dependencies()</code>              | "Start pushing dep tokens to downstream modules"                |
| <code>dep_pushed</code>             | <code>push_prev/next_end</code> handlers                                 | <code>dependencies_pushed()</code>            | "A dep token was accepted — check if all are done"              |
| <code>send_signal</code>            | <code>finalize_instruction()</code> on FINISH                            | <code>send_signal_handler()</code>            | "Pulse out_trig to ARM"                                         |
| <code>activate_push_prev_vld</code> | <code>push_dependencies()</code>                                         | <code>activate_push_prev_vld_handler()</code> | "Write push_prev_vld port now"                                  |
| <code>activate_push_prev_end</code> | <code>write_push_prev_data</code> handler                                | <code>activate_push_prev_end_handler()</code> | "Write push_prev_end port now"                                  |
| <code>activate_push_next_vld</code> | <code>push_dependencies()</code>                                         | <code>activate_push_next_vld_handler()</code> | "Write push_next_vld port now"                                  |
| <code>activate_push_next_end</code> | <code>write_push_next_data</code> handler                                | <code>activate_push_next_end_handler()</code> | "Write push_next_end port now"                                  |
| <code>activate_pull_prev_rdy</code> | <code>check/receive_dependencies()</code>                                | <code>activate_pull_prev_rdy_handler()</code> | "Write pull_prev_rdy port now"                                  |

|                                     |                                                |                                               |                                                       |
|-------------------------------------|------------------------------------------------|-----------------------------------------------|-------------------------------------------------------|
| <code>activate_pull_next_rdy</code> | <code>check/receive_dependencies()</code>      | <code>activate_pull_next_rdy_handler()</code> | "Write pull_next_rdy port now"                        |
| <code>write_push_prev_data</code>   | <code>push_prev_rdy_handler (rdy rises)</code> | <code>write_push_prev_data_handler()</code>   | "Queue accepted our VLD — write the data now"         |
| <code>read_pull_prev_data</code>    | <code>activate_pull_prev_rdy_handler</code>    | <code>read_pull_prev_data_handler()</code>    | "We set RDY — read the data that queue is presenting" |
| <code>write_push_next_data</code>   | <code>push_next_rdy_handler</code>             | <code>write_push_next_data_handler()</code>   | "c2s queue accepted — write data"                     |
| <code>read_pull_next_data</code>    | <code>activate_pull_next_rdy_handler</code>    | <code>read_pull_next_data_handler()</code>    | "Read data from s2c queue"                            |

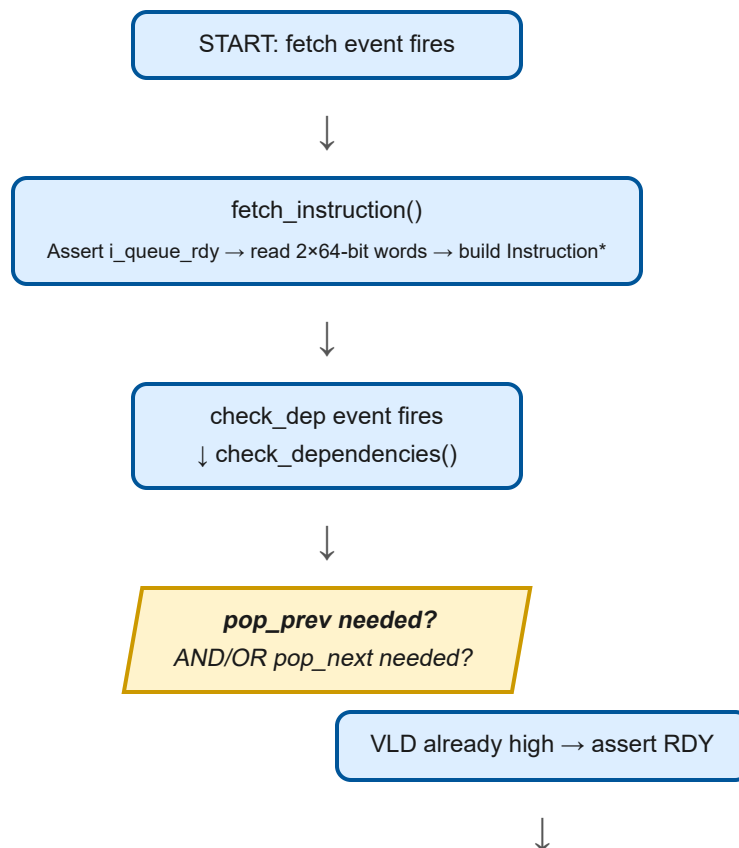
## 4. Dependency Queue Names: HLS vs SystemC

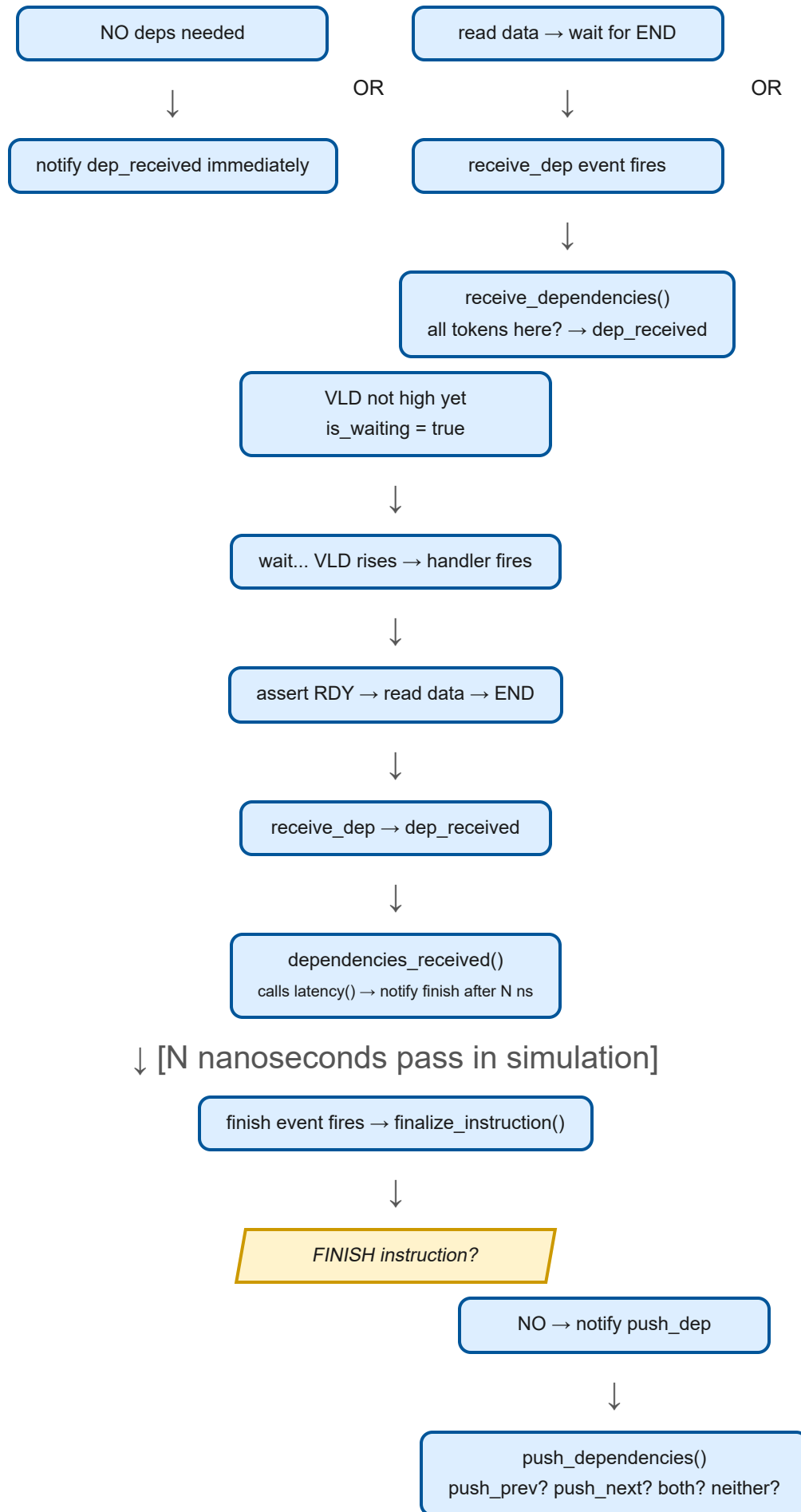
| HLS name in vta.cc         | SystemC name in simulator                             | Direction       | Meaning                                                       |
|----------------------------|-------------------------------------------------------|-----------------|---------------------------------------------------------------|
| <code>l2g_dep_queue</code> | <code>l2c_queue</code> / <code>pull_prev</code> ports | Load → Compute  | Load finished loading inputs/weights — Compute can now start  |
| <code>g2l_dep_queue</code> | <code>c2l_queue</code> / <code>push_prev</code> ports | Compute → Load  | Compute is done with the input buffer — Load can overwrite it |
| <code>s2g_dep_queue</code> | <code>s2c_queue</code> / <code>pull_next</code> ports | Store → Compute | Store cleared out_mem — Compute can write new results         |
| <code>g2s_dep_queue</code> | <code>c2s_queue</code> / <code>push_next</code> ports | Compute → Store | Compute wrote results to out_mem — Store can now copy to DRAM |

**Memory:** HLS used "g" for GEMM stage. SystemC uses "c" for Compute stage. Same concept, renamed to avoid confusion with the instruction type GEMM.

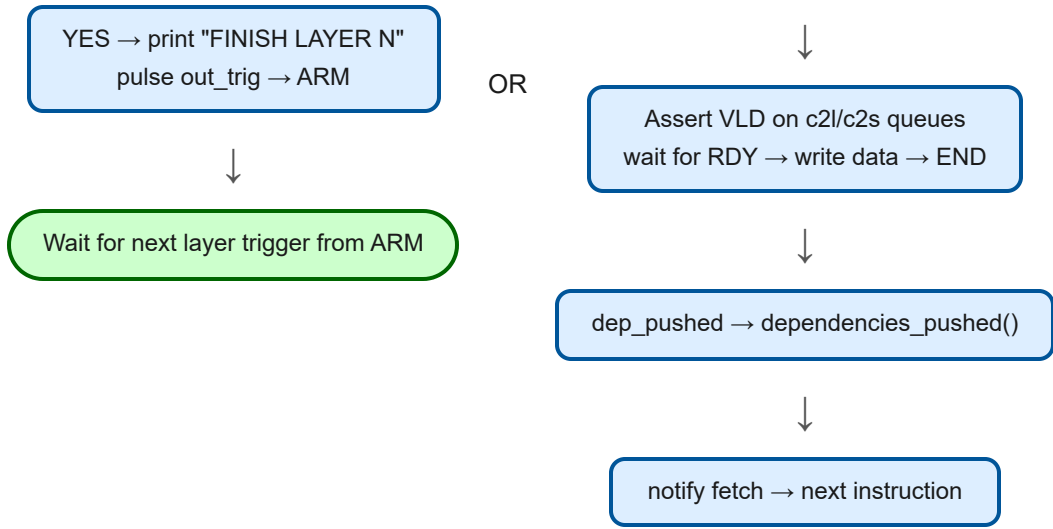
## 5. Dataflow Flowcharts

### 5.1 Top-Level Lifecycle of One Instruction









5.2 Dependency Checking Detail (check\_dependencies)

This mirrors the dependency check in `vta.cc compute()` :

HLS:

```
if (insn.generic.pop_prev_dep) { l2g_dep_queue.read(); } // blocks until token available
if (insn.generic.pop_next_dep) { s2g_dep_queue.read(); } // blocks until token available
```

SystemC (can't block — must use events):

```
void ComputeModule::check_dependencies() {
    if (pop_prev && !pop_next) {
        if (pull_prev_vld.read()) { // token already waiting?
            pull_prev_rdy_state = true; // yes - accept it immediately
            activate_pull_prev_rdy.notify(1, SC_NS);
        } else {
            is_waiting_prev = true; // no - set flag and wait
        } // pull_prev_vld_handler() will fire when it arrives
    }
    // ... similar for pop_next, and both together
    else { dep_received.notify(SC_ZERO_TIME); } // no deps needed - proceed
}
```

5.3 Latency Calculation (what replaces GEMM/ALU math)

HLS runs the actual dot-product loop. SystemC replaces it with a time model:

| Instruction type | Latency formula                                                                                    | Variables used                                                    |
|------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| GEMM             | $(range\_1 - range\_0) \times outer\_loop\_iter \times inner\_loop\_iter + 100\text{ ns}$          | range_0/1 = uop range; outer/inner = loop bounds from instruction |
| ALU              | $2 \times (range\_1 - range\_0) \times outer\_loop\_iter \times inner\_loop\_iter + 100\text{ ns}$ | Factor 2 because HLS pipeline II=2 for ALU (vs II=1 for GEMM)     |

|                          |                                                                                                            |                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| LOAD ACC<br>(y_size≠0)   | $y\_size \times 100 + load\_cycles \times \lceil y\_size \times x\_size \times 64 / 256 \rceil \text{ ns}$ | Loading accumulator bias tiles into on-chip SRAM |
| Other LOAD<br>(y_size≠0) | $y\_size \times 100 + load\_cycles \times \lceil y\_size \times x\_size \times 16 / 256 \rceil \text{ ns}$ | load_cycles from config.json                     |
| NOP-COMPUTE-STAGE        | 15 ns                                                                                                      | —                                                |
| FINISH                   | 10 ns                                                                                                      | —                                                |

**Why range\_1 - range\_0?** In HLS, `uop_bgn` and `uop_end` define the micro-op loop range — how many micro-ops (each doing one accumulator tile operation) are executed. `range_0 = uop_bgn`, `range_1 = uop_end`. The difference is the number of micro-ops. Multiplied by the outer and inner loop iteration counts gives total compute steps.

## 5.4 Valid/Ready Handshake Protocol (for every queue interaction)

This is the hardware protocol used for all queue communication (instruction queue AND dependency queues):

|                                                                                                                                        |                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Sender side:</p> <pre> assert VLD = 1 (data available on bus)  (wait for RDY = 1) write DATA on bus assert END = 1           </pre> | <p>Receiver side:</p> <pre> sees VLD = 1 → assert RDY = 1 (wait 1 ns) read DATA from bus assert END = 1 ← signals "I read it" (wait 1 ns) de-assert RDY = 0           </pre> |
|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Timeline: VLD↑ ... RDY↑ ... DATA read ... END↑ ... VLD↓ RDY↓

Every Queue module (instruction queues l2c, c2l, c2s, s2c) uses these four signals: `vld` (sender has data), `rdy` (receiver is ready), `data` (payload), `end` (transaction complete).

## 6. Complete SystemC Code Listing

### 6.1 compute\_module.h

```

#ifndef COMPUTE_MODULE_H
#define COMPUTE_MODULE_H

#include <systemc.h>
#include <queue>
#include "module.h"          // abstract base: fetch, check_dep, events, etc.

class ComputeModule : public Module {
public:
    // — Ports —————

    // Trigger to ARM: HIGH for one tick when a FINISH instruction completes
    sc_out<bool> out_trig;

    // Pull-prev: receive dependency token FROM l2c queue (Load→Compute)
    sc_in<bool>      pull_prev_vld;    // l2c says: token ready
    sc_out<bool>      pull_prev_rdy;    // we say: we accept it
    sc_in<sc_int<64>> pull_prev_data;  // token payload (Load's PC)
    sc_in<bool>      pull_prev_end;    // handshake complete

    // Push-prev: send dependency token TO c2l queue (Compute→Load)
    sc_out<bool>      push_prev_vld;    // we say: token ready
    sc_in<bool>      push_prev_rdy;    // c2l says: it accepts
    sc_out<sc_int<64>> push_prev_data;  // token payload (our PC)
    sc_out<bool>      push_prev_end;    // handshake complete

    // Pull-next: receive dependency token FROM s2c queue (Store→Compute)
    sc_in<bool>      pull_next_vld;
    sc_out<bool>      pull_next_rdy;
    sc_in<sc_int<64>> pull_next_data;
    sc_in<bool>      pull_next_end;

    // Push-next: send dependency token TO c2s queue (Compute→Store)
    sc_out<bool>      push_next_vld;
    sc_in<bool>      push_next_rdy;
    sc_out<sc_int<64>> push_next_data;
    sc_out<bool>      push_next_end;

    SC_HAS_PROCESS(ComputeModule);
    ComputeModule(sc_module_name n);

    static int current_layer;    // global layer counter, printed on FINISH

private:
    // — Received dependency token payloads —————
    sc_int<64> *prev_data = nullptr;  // from l2c (Load→Compute token payload)
    sc_int<64> *next_data = nullptr;  // from s2c (Store→Compute token payload)

    // — Waiting flags (set when dep needed but token not yet arrived) ———
    bool is_waiting_prev = false;
    bool is_waiting_next = false;

    // — Output signal staging —————
    bool out_signal = false;         // desired value for out_trig

```

```

// — Dep-push completion tracking —————
bool did_push_prev = false;      // c2l token handshake completed
bool did_push_next = false;      // c2s token handshake completed

// — Port state staging (actual port written by handler, not directly) ———
bool push_prev_vld_state = false;
bool push_prev_end_state = false;
bool push_next_vld_state = false;
bool push_next_end_state = false;
bool pull_next_rdy_state = false;
bool pull_prev_rdy_state = false;

// — Internal events —————
sc_event send_signal;            // fire to pulse out_trig

sc_event activate_push_prev_vld; // fire to write push_prev_vld port
sc_event activate_push_prev_end; // fire to write push_prev_end port
sc_event activate_push_next_vld; // fire to write push_next_vld port
sc_event activate_push_next_end; // fire to write push_next_end port
sc_event activate_pull_prev_rdy;  // fire to write pull_prev_rdy port
sc_event activate_pull_next_rdy;  // fire to write pull_next_rdy port

sc_event write_push_prev_data;    // fire to write data onto push_prev_data port
sc_event read_pull_prev_data;    // fire to read data from pull_prev_data port
sc_event write_push_next_data;
sc_event read_pull_next_data;

// — Virtual method overrides from Module base —————
sc_time latency() override;
void check_dependencies() override;
void receive_dependencies() override;
void dependencies_received() override;
void finalize_instruction() override;
void push_dependencies() override;
void dependencies_pushed() override;

// — SC_METHOD handlers —————
void send_signal_handler();

void activate_push_prev_vld_handler();
void activate_push_prev_end_handler();
void activate_push_next_vld_handler();
void activate_push_next_end_handler();
void activate_pull_prev_rdy_handler();
void activate_pull_next_rdy_handler();

void pull_prev_vld_handler();
void pull_prev_end_handler();
void read_pull_prev_data_handler();

void push_prev_rdy_handler();
void write_push_prev_data_handler();

void pull_next_vld_handler();
void pull_next_end_handler();
void read_pull_next_data_handler();

void push_next_rdy_handler();
void write_push_next_data_handler();
};

#endif

```

## 6.2 compute\_module.cpp — Constructor

```
#include "compute_module.h"

int ComputeModule::current_layer = 0;    // starts at 0, increments on each FINISH

ComputeModule::ComputeModule(sc_module_name n) : Module(n) {

    // — Wire up every internal event to its SC_METHOD handler —————
    // send_signal fires when FINISH instruction completes → pulse out_trig
    SC_METHOD(send_signal_handler);
    dont_initialize();
    sensitive << send_signal;

    // Port-write handlers: fire → write the staging variable to the real port
    SC_METHOD(activate_push_prev_vld_handler);
    dont_initialize();
    sensitive << activate_push_prev_vld;

    SC_METHOD(activate_push_prev_end_handler);
    dont_initialize();
    sensitive << activate_push_prev_end;

    SC_METHOD(activate_push_next_vld_handler);
    dont_initialize();
    sensitive << activate_push_next_vld;

    SC_METHOD(activate_push_next_end_handler);
    dont_initialize();
    sensitive << activate_push_next_end;

    SC_METHOD(activate_pull_prev_rdy_handler);
    dont_initialize();
    sensitive << activate_pull_prev_rdy;

    SC_METHOD(activate_pull_next_rdy_handler);
    dont_initialize();
    sensitive << activate_pull_next_rdy;

    // Data read/write handlers
    SC_METHOD(write_push_prev_data_handler);
    dont_initialize();
    sensitive << write_push_prev_data;

    SC_METHOD(write_push_next_data_handler);
    dont_initialize();
    sensitive << write_push_next_data;

    SC_METHOD(read_pull_prev_data_handler);
    dont_initialize();
    sensitive << read_pull_prev_data;

    SC_METHOD(read_pull_next_data_handler);
    dont_initialize();
    sensitive << read_pull_next_data;

    // Incoming signal handlers (react to port changes from outside modules)
    SC_METHOD(pull_prev_vld_handler);    // fires when l2c queue raises VLD
    dont_initialize();
    sensitive << pull_prev_vld.pos();    // .pos() = positive edge only

    SC_METHOD(pull_prev_end_handler);
```

```

dont_initialize();
sensitive << pull_prev_end.pos();

SC_METHOD(push_prev_rdy_handler); // fires when c2l queue raises RDY
dont_initialize();
sensitive << push_prev_rdy.pos();

SC_METHOD(pull_next_vld_handler); // fires when s2c queue raises VLD
dont_initialize();
sensitive << pull_next_vld.pos();

SC_METHOD(pull_next_end_handler);
dont_initialize();
sensitive << pull_next_end.pos();

SC_METHOD(push_next_rdy_handler); // fires when c2s queue raises RDY
dont_initialize();
sensitive << push_next_rdy.pos();
}

```

### 6.3 compute\_module.cpp — latency()

```

// Called by dependencies_received() to know how long to wait before finalize_instruction()
// This is the SystemC replacement for the actual GEMM/ALU math loops in HLS.
// We don't compute the result – we compute how long it would take in real hardware.
sc_time ComputeModule::latency() {
    int inst_exec_time = 15; // default NOP time

    if (current->get_y_size() != 0) {
        // LOAD ACC or other memory-type compute instruction
        if (current->get_name() == "LOAD ACC") {
            // Accumulator tile load: 64 bits per element
            inst_exec_time = current->get_y_size() * 100
                + PlatformConfig::getInstance().load_cycles
                * std::ceil(current->get_y_size() * current->get_x_size() * 64.0 / 256.0);
        } else {
            // Other LOAD in compute stage: 16 bits per element
            inst_exec_time = current->get_y_size() * 100
                + PlatformConfig::getInstance().load_cycles
                * std::ceil(current->get_y_size() * current->get_x_size() * 16.0 / 256.0);
        }
    } else {
        if (current->get_name() == "GEMM") {
            // HLS pipeline II=1, uop range × outer_iter × inner_iter cycles
            inst_exec_time = (current->get_range_1() - current->get_range_0())
                * current->get_outer_loop_iter()
                * current->get_inner_loop_iter()
                + 100; // +100 ns startup overhead
        }
        else if (current->get_name() == "NOP-COMPUTE-STAGE") {
            inst_exec_time = 15;
        }
        else if (current->get_name() == "FINISH") {
            inst_exec_time = 10;
        }
        else {
            // ALU (ReLU, MAX, ADD, SHR): HLS pipeline II=2, so ×2
            inst_exec_time = 2 * (current->get_range_1() - current->get_range_0())
                * current->get_outer_loop_iter()
                * current->get_inner_loop_iter()
                + 100;
        }
    }
}

```

```

    }
}
return sc_time(inst_exec_time, SC_NS);
}

```

## 6.4 compute\_module.cpp — Dependency Checking and Receiving

```

// Called when a new instruction arrives.
// Checks whether dependency tokens (from Load or Store) are needed.
void ComputeModule::check_dependencies() {
    if (current->get_pop_prev() && !current->get_pop_next()) {
        // Need l2c token (Load→Compute) only
        if (this->pull_prev_vld.read()) { // token already on the queue?
            this->pull_prev_rdy_state = true;
            activate_pull_prev_rdy.notify(1, SC_NS); // accept it
        } else {
            this->is_waiting_prev = true; // not yet – flag and wait
        }
    }
    else if (!current->get_pop_prev() && current->get_pop_next()) {
        // Need s2c token (Store→Compute) only
        if (this->pull_next_vld.read()) {
            this->pull_next_rdy_state = true;
            activate_pull_next_rdy.notify(1, SC_NS);
        } else {
            this->is_waiting_next = true;
        }
    }
    else if (current->get_pop_prev() && current->get_pop_next()) {
        // Need BOTH tokens – check each independently
        if (this->pull_prev_vld.read()) {
            this->pull_prev_rdy_state = true;
            activate_pull_prev_rdy.notify(1, SC_NS);
        } else {
            this->is_waiting_prev = true;
        }
        if (this->pull_next_vld.read()) {
            this->pull_next_rdy_state = true;
            activate_pull_next_rdy.notify(1, SC_NS);
        } else {
            this->is_waiting_next = true;
        }
    }
    else {
        // No deps needed – immediately proceed to execution
        this->dep_received.notify(SC_ZERO_TIME);
    }
}

// Called after each dep token END signal fires.
// Checks if ALL required tokens have now been received.
void ComputeModule::receive_dependencies() {
    if (current->get_pop_prev() && !current->get_pop_next() && this->prev_data != nullptr) {
        this->dep_received.notify(SC_ZERO_TIME); // only needed prev – got it
    }
    else if (!current->get_pop_prev() && current->get_pop_next() && this->next_data != nullptr) {
        this->dep_received.notify(SC_ZERO_TIME); // only needed next – got it
    }
    else if (current->get_pop_prev() && current->get_pop_next()
             && this->prev_data != nullptr && this->next_data != nullptr) {
        this->dep_received.notify(SC_ZERO_TIME); // needed both – both received
    }
}

```

```

    // if one of the required tokens hasn't arrived yet, do nothing – wait for
    // the other pull_prev_end or pull_next_end to fire receive_dependencies again
}

// Called when dep_received fires (all tokens in).
// Starts the simulated execution timer.
void ComputeModule::dependencies_received() {
    finish.notify(latency());    // "wake me up after N nanoseconds"
}

```

## 6.5 compute\_module.cpp — Finalize and Push Dependencies

```

// Called when finish event fires (simulated time elapsed).
void ComputeModule::finalize_instruction() {
    this->result_data = new sc_int<32>(5);    // placeholder result

    if (this->current->get_name() == "FINISH") {
        // Layer boundary – print timing and signal ARM
        std::cout << sc_time_stamp() << "\t\t"
            << "----- FINISH LAYER " << ComputeModule::current_layer++ << " -----"
            << std::endl;

        // Clean up heap allocations
        if (result_data != nullptr) delete result_data;
        if (prev_data != nullptr) delete prev_data;
        if (next_data != nullptr) delete next_data;
        delete current;

        this->current = nullptr;
        this->result_data = nullptr;
        this->prev_data = nullptr;
        this->next_data = nullptr;
        this->did_push_prev = false;
        this->did_push_next = false;

        // Signal ARM: this layer is done
        this->out_signal = true;
        this->send_signal.notify(SC_ZERO_TIME);
    } else {
        // Normal instruction – push dep tokens to downstream if required
        this->push_dep.notify(SC_ZERO_TIME);
    }
}

// Decides which dep tokens to push (c2l, c2s, both, or neither).
void ComputeModule::push_dependencies() {
    if (current->get_push_prev() && !current->get_push_next()) {
        // Push c2l only (tell Load it can reuse input buffer)
        this->push_prev_vld_state = true;
        this->activate_push_prev_vld.notify(1, SC_NS);
    }
    else if (!current->get_push_prev() && current->get_push_next()) {
        // Push c2s only (tell Store it can copy results)
        this->push_next_vld_state = true;
        this->activate_push_next_vld.notify(1, SC_NS);
    }
    else if (current->get_push_prev() && current->get_push_next()) {
        // Push both
        this->push_prev_vld_state = true;
        this->activate_push_prev_vld.notify(1, SC_NS);
        this->push_next_vld_state = true;
    }
}

```



```

        this->activate_push_next_vld.notify(1, SC_NS);
    } else {
        // No deps to push – clean up and fetch next instruction
        if (result_data != nullptr) delete result_data;
        if (prev_data != nullptr) delete prev_data;
        if (next_data != nullptr) delete next_data;
        delete current;
        this->current = nullptr;
        this->result_data = nullptr;
        this->prev_data = nullptr;
        this->next_data = nullptr;
        this->did_push_prev = false;
        this->did_push_next = false;
        this->fetch.notify(1, SC_NS);    // go get next instruction
    }
}

// Called after each dep_pushed event – checks if all required pushes are done.
void ComputeModule::dependencies_pushed() {
    bool prev_done = !current->get_push_prev() || this->did_push_prev;
    bool next_done = !current->get_push_next() || this->did_push_next;
    if (prev_done && next_done) {
        // All required tokens sent – clean up and fetch next instruction
        if (result_data != nullptr) delete result_data;
        if (prev_data != nullptr) delete prev_data;
        if (next_data != nullptr) delete next_data;
        delete current;
        this->current = nullptr;
        this->result_data = nullptr;
        this->prev_data = nullptr;
        this->next_data = nullptr;
        this->did_push_prev = false;
        this->did_push_next = false;

        // De-assert VLD lines (handshake complete)
        this->push_prev_vld_state = false;
        this->activate_push_prev_vld.notify(1, SC_NS);
        this->push_next_vld_state = false;
        this->activate_push_next_vld.notify(1, SC_NS);

        this->fetch.notify(1, SC_NS);
    }
}

```

## 6.6 compute\_module.cpp — Signal Handlers

```

// — ARM trigger pulse —————

void ComputeModule::send_signal_handler() {
    this->out_trig.write(this->out_signal);
    if (this->out_signal) {
        this->out_signal = false;    // de-assert after 1 ns
        this->send_signal.notify(1, SC_NS);    // schedule de-assert
    }
}

// — Port write handlers —————

void ComputeModule::activate_push_prev_vld_handler() {
    this->push_prev_vld.write(this->push_prev_vld_state);
}

```

```

void ComputeModule::activate_push_prev_end_handler() {
    this->push_prev_end.write(this->push_prev_end_state);
    if (push_prev_end_state) {
        this->push_prev_end_state = false;
        this->activate_push_prev_end.notify(1, SC_NS); // de-assert after 1 ns
        this->did_push_prev = true;
        this->dep_pushed.notify(SC_ZERO_TIME);
    }
}

void ComputeModule::activate_push_next_vld_handler() {
    this->push_next_vld.write(this->push_next_vld_state);
}

void ComputeModule::activate_push_next_end_handler() {
    this->push_next_end.write(this->push_next_end_state);
    if (this->push_next_end_state) {
        this->push_next_end_state = false;
        this->activate_push_next_end.notify(1, SC_NS);
        this->did_push_next = true;
        this->dep_pushed.notify(SC_ZERO_TIME);
    }
}

void ComputeModule::activate_pull_prev_rdy_handler() {
    this->pull_prev_rdy.write(this->pull_prev_rdy_state);
    if (this->pull_prev_rdy_state) {
        read_pull_prev_data.notify(1, SC_NS); // read data 1 ns after asserting RDY
    }
}

void ComputeModule::activate_pull_next_rdy_handler() {
    this->pull_next_rdy.write(this->pull_next_rdy_state);
    if (this->pull_next_rdy_state) {
        read_pull_next_data.notify(1, SC_NS);
    }
}

// — Incoming token handlers —————

// pull_prev_vld rises: l2c token arrived while we were waiting
void ComputeModule::pull_prev_vld_handler() {
    if (current && current->get_pop_prev() && this->is_waiting_prev) {
        this->is_waiting_prev = false;
        this->pull_prev_rdy_state = true;
        activate_pull_prev_rdy.notify(1, SC_NS); // now accept it
    }
}

void ComputeModule::read_pull_prev_data_handler() {
    // Store the token payload (Load's PC) – used to confirm receipt
    this->prev_data = new sc_int<64>(this->pull_prev_data.read());
}

void ComputeModule::pull_prev_end_handler() {
    // Handshake complete – de-assert RDY and check if all deps received
    this->pull_prev_rdy_state = false;
    activate_pull_prev_rdy.notify(SC_ZERO_TIME);
    this->receive_dep.notify(SC_ZERO_TIME);
}

// push_prev_rdy rises: c2l queue ready to receive our c2l token

```

```
void ComputeModule::push_prev_rdy_handler() {
    write_push_prev_data.notify(SC_ZERO_TIME); // write data now
}

void ComputeModule::write_push_prev_data_handler() {
    this->push_prev_data.write(this->current->get_pc()); // our PC as payload
    this->push_prev_end_state = true;
    this->activate_push_prev_end.notify(1, SC_NS); // signal end of handshake
}

// pull_next_vld rises: s2c token arrived while waiting
void ComputeModule::pull_next_vld_handler() {
    if (current && current->get_pop_next() && this->is_waiting_next) {
        this->is_waiting_next = false;
        this->pull_next_rdy_state = true;
        activate_pull_next_rdy.notify(1, SC_NS);
    }
}

void ComputeModule::read_pull_next_data_handler() {
    this->next_data = new sc_int<64>(this->pull_next_data.read());
}

void ComputeModule::pull_next_end_handler() {
    this->pull_next_rdy_state = false;
    activate_pull_next_rdy.notify(1, SC_NS);
    this->receive_dep.notify(SC_ZERO_TIME);
}

// push_next_rdy rises: c2s queue ready to receive our c2s token
void ComputeModule::push_next_rdy_handler() {
    write_push_next_data.notify(SC_ZERO_TIME);
}

void ComputeModule::write_push_next_data_handler() {
    this->push_next_data.write(this->current->get_pc());
    this->push_next_end_state = true;
    this->activate_push_next_end.notify(1, SC_NS);
}
```

## 7. Worked Example: Tracing One GEMM Instruction

### 7.1 The Dummy Instruction (as it would appear in the CSV / Instruction object)

| Field             | Value     | Meaning                                          |
|-------------------|-----------|--------------------------------------------------|
| name              | "GEMM"    | Matrix multiply                                  |
| pc                | 42        | Program counter — this is instruction #42        |
| layer             | "layer_5" | 5th layer of the neural network                  |
| pop_prev          | 1         | Must wait for l2c token (Load must finish first) |
| pop_next          | 0         | No need to wait for s2c token                    |
| push_prev         | 0         | Does NOT send c2l token after                    |
| push_next         | 1         | WILL send c2s token (Store can proceed after)    |
| range_0 (uop_bgn) | 0         | Micro-op loop starts at index 0                  |
| range_1 (uop_end) | 4         | Micro-op loop ends at index 4 → 4 micro-ops      |
| outer_loop_iter   | 3         | Outer loop runs 3 times                          |
| inner_loop_iter   | 2         | Inner loop runs 2 times                          |
| y_size            | 0         | Not a memory-type instruction (GEMM path)        |

#### Latency calculation:

```

GEMM latency = (range_1 - range_0) × outer_loop_iter × inner_loop_iter + 100
               = (4 - 0) × 3 × 2 + 100
               = 4 × 6 + 100
               = 24 + 100
               = 124 ns

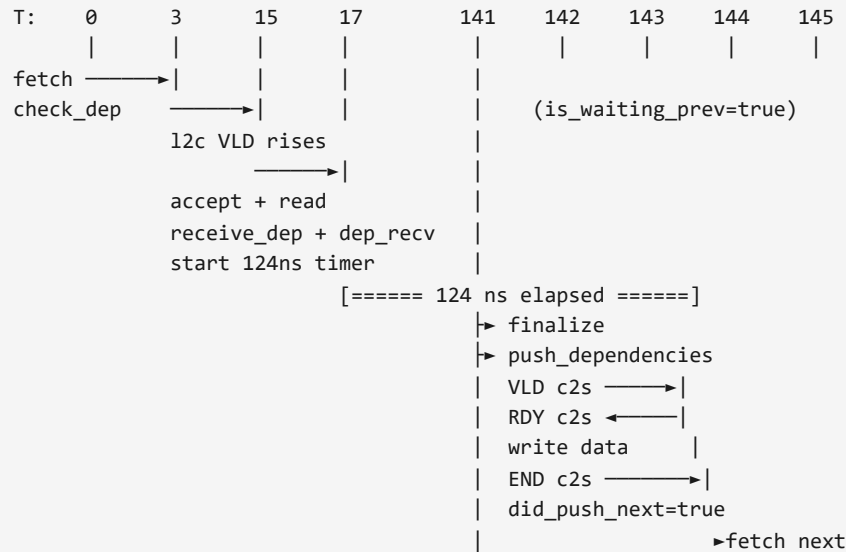
```

### 7.2 Step-by-Step Event Trace

| Time   | Event / Method                 | What happens                                                                                                                     | State after                              |
|--------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| T+0 ns | <code>fetch</code> event fires | <code>fetch_instruction()</code> begins. Asserts <code>i_queue_rdy</code> . Waits for compute instruction queue VLD.             | <code>i_queue_rdy=1</code> , waiting     |
| T+2 ns | <code>i_queue_vld</code> rises | Queue has instruction. <code>read_i_queue_data</code> fires. First 64-bit word read into <code>current_instruction_part</code> . | 1 word received                          |
| T+3 ns | <code>i_queue_end</code> rises | Instruction complete (2 words received). Builds <code>Instruction*</code> object. Notifies                                       | <code>current = Instruction(GEMM,</code> |

|          |                                       |                                                                                                                             |                                 |
|----------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------|
|          |                                       | check_dep.                                                                                                                  | pc=42)                          |
| T+3 ns   | check_dependencies()                  | pop_prev=1, pop_next=0. Reads pull_prev_vld. Suppose I2c token NOT yet present → is_waiting_prev = true                     | is_waiting_prev=true            |
| T+15 ns  | pull_prev_vld rises                   | I2c queue now has a token. pull_prev_vld_handler fires. is_waiting_prev was true → clear flag, set pull_prev_rdy_state=true | is_waiting_prev=false           |
| T+16 ns  | activate_pull_prev_rdy_handler        | Writes pull_prev_rdy=1 to port. Schedules read_pull_prev_data after 1 ns.                                                   | pull_prev_rdy port = 1          |
| T+17 ns  | read_pull_prev_data_handler           | Reads pull_prev_data (token payload = Load's PC, say 38). Stores in prev_data = new sc_int<64>(38).                         | prev_data = 38                  |
| T+17 ns  | pull_prev_end rises                   | I2c queue says handshake complete. pull_prev_end_handler fires: sets pull_prev_rdy=0, notifies receive_dep.                 | pull_prev_rdy=0                 |
| T+17 ns  | receive_dependencies()                | pop_prev=1, pop_next=0, prev_data≠nullptr → all satisfied → notifies dep_received.                                          | —                               |
| T+17 ns  | dependencies_received()               | Calls latency() → 124 ns. Calls finish.notify(124 ns).                                                                      | finish scheduled at T+141       |
| T+141 ns | finish fires → finalize_instruction() | name="GEMM" (not FINISH) → notifies push_dep.                                                                               | result_data = new sc_int<32>(5) |
| T+141 ns | push_dependencies()                   | push_prev=0, push_next=1 → set push_next_vld_state=true, notify activate_push_next_vld in 1 ns.                             | —                               |
| T+142 ns | activate_push_next_vld_handler        | Writes push_next_vld=1 to port (c2s queue). c2s queue detects VLD rising.                                                   | push_next_vld=1                 |
| T+143 ns | push_next_rdy rises (from c2s queue)  | push_next_rdy_handler fires → notifies write_push_next_data.                                                                | —                               |
| T+143 ns | write_push_next_data_handler          | Writes push_next_data = 42 (our PC). Sets push_next_end_state=true, notifies activate_push_next_end in 1 ns.                | push_next_data=42               |
| T+144 ns | activate_push_next_end_handler        | Writes push_next_end=1. did_push_next=true. Notifies dep_pushed.                                                            | did_push_next=true              |
| T+144 ns | dependencies_pushed()                 | push_prev needed? No. push_next needed? Yes, and did_push_next=true → both done. Clean up, notify fetch in 1 ns.            | current=nullptr, all cleaned    |
| T+145 ns | fetch fires                           | Fetch next instruction (pc=43). Cycle repeats.                                                                              | —                               |

### 7.3 Visual Timeline



## 7.4 What This Tells Us About the Network

If many consecutive GEMM instructions each take 124 ns, and a layer has 50 such instructions, that layer's compute time is roughly  $50 \times 124 = 6200 \text{ ns}$ . The "FINISH LAYER 5" line would print at about 6.2  $\mu\text{s}$  after the layer started (plus dependency wait times). This is exactly the kind of performance data the simulator is designed to produce — so you can study how layer size, loop bounds, and latency config affect total runtime.

## 8. Summary: What You Need to Remember

| Key idea                   | One-sentence explanation                                                                                                                                                          |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No blocking in SystemC     | HLS functions block until deps arrive; SystemC sets <code>is_waiting_*</code> flags and reacts to signal edges instead.                                                           |
| Events are the clock       | Every action is triggered by an <code>sc_event</code> ; methods run zero-duration unless they schedule a future notify (e.g. <code>finish.notify(124 ns)</code> ).                |
| Simulator doesn't compute  | GEMM/ALU math is replaced by <code>latency()</code> which returns how long the hardware would take. No actual dot products are computed.                                          |
| Token = permission slip    | A dep token is not data — it's a "you can go now" signal. Its payload (a PC value) is just a tag to identify which instruction sent it.                                           |
| Two handshakes per token   | Sending a token requires: assert VLD → wait for RDY → write data → assert END. Receiving: see VLD → assert RDY → read data → see END.                                             |
| FINISH instruction special | FINISH does not push dep tokens. It pulses out_trig → ARM, ARM starts the next layer. <code>current_layer++</code> happens here.                                                  |
| State variables reset      | After every instruction (FINISH or dep-push done), <code>current/prev_data/next_data</code> are deleted and set to <code>nullptr</code> , <code>did_push_*</code> reset to false. |